Programming C – Module: Memory Management and Recursion

Why Memory Management Matters

- Every C program uses memory to store:
  - Variables
  - Function calls
  - Data structures
- Understanding memory helps:
  - Avoid errors (crashes, leaks)
  - Write efficient programs
  - Understand program behavior

Memory Layout of a C Program

Typical memory areas:

- **Code (text segment)** → program instructions
- **Global/static area** → global & static variables
- **Stack** → function calls and local variables
- **Heap** → dynamic memory (malloc, free)

Global Variables

PROF

- Declared **outside all functions**
- Accessible from any function

```c
int globalVar = 10;

void func() {
    printf("%d", globalVar);
}
```

Properties:

- Stored in **global/static area**
- Lifetime = entire program execution
- Automatically initialized (default = 0)

Exporting Global Variables (`extern`)

- Global variables can be **shared across multiple files**

Declaration in one file:

```
int globalVar = 10;
```

Use in another file:

```
extern int globalVar;
```

Notes:

- `extern` tells the compiler:
  “this variable is defined elsewhere”
- Avoids multiple definitions
- Enables modular program design

Static Variables

Static Local Variable

```
void counter() {
    static int count = 0;
    count++;
    printf("%d\n", count);
}
```

PROF

- Retains value between function calls

Properties:

- Stored in **global/static area**
- Lifetime = entire program
- Scope = limited to function

Global vs Static

Feature	Global	Static (local)
Scope	Entire program	Function/block

Feature	Global	Static (local)
Lifetime	Entire program	Entire program
Visibility	Everywhere	Limited

Local (Automatic) Variables

- Declared inside functions

```
void func() {
    int x = 5;
}
```

Properties:

- Stored in **stack**
- Lifetime = during function execution
- Created when function starts
- Destroyed when function ends

Stack Memory

- Used for:
    - Function calls
    - Local variables
- Organized as **LIFO (Last In, First Out)**

 Each function call creates a **stack frame**

PROF

Function Call and Stack Frames

Example:

```
void f2() { }
void f1() { f2(); }

int main() {
    f1();
}
```

Execution:

1. `main()` pushed
2. `f1()` pushed
3. `f2()` pushed
4. `f2()` returns → popped
5. `f1()` returns → popped

Stack Frame Contents

Each function call stores:

- Parameters
- Local variables
- Return address

 Enables function execution and return flow

Example: Stack Behavior

```c
void func() {
    int x = 10;
}

int main() {
    func();
}
```

- `x` exists only during `func()`
- After return → memory is released automatically

PROF

Static vs Automatic (Detailed Example)

```c
void test() {
    static int x = 0;
    int y = 0;

    x++;
    y++;

    printf("x=%d y=%d\n", x, y);
}
```

Call sequence:

```
```

```
test();
test();
test();
```

Step-by-step behavior:

First call:

- x (static) → initialized once → becomes 1
- y (local) → created → becomes 1
  Output:

```
x=1 y=1
```

Second call:

- x → keeps previous value → becomes 2
- y → recreated → starts again from 0 → becomes 1
  Output:

```
x=2 y=1
```

Third call:

- x → becomes 3
- y → again reset → becomes 1
  Output:


```
x=3 y=1
```

Key Insight:

- x is stored in **static/global memory** → persists across calls
- y is stored in **stack memory** → recreated each call

 This clearly shows the difference between:

- **Static lifetime (persistent)**
- **Automatic lifetime (temporary, stack-based)**

Summary

- Static variables persist across function calls
- Local variables exist only during execution
- Stack handles temporary data
- Understanding this helps predict program behavior

Recursive Functions

- A function that calls itself

```
int factorial(int n) {
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}
```

Recursion and Stack

Call:

```
factorial(3);
```

Stack grows:

- factorial(3)
- factorial(2)
- factorial(1)
- factorial(0)

Then unwinds:

- returns 1 → 1 → 2 → 6

Visualizing Recursion

```
factorial(3)
  → 3 * factorial(2)
      → 2 * factorial(1)
          → 1 * factorial(0)
              → 1
```

Base Case Importance

- Prevents infinite recursion

```
if (n == 0)
    return 1;
```

⚠ Missing base case → stack overflow

Stack Overflow

- Occurs when stack memory is exhausted

Causes:

- Infinite recursion
- Too many nested calls
- Large local variables

Why Recursion Matters?

- Recursion is a technique where a function **calls itself**
- It is useful for solving problems that can be broken into **smaller subproblems**
- Often leads to **simpler and clearer solutions**

Core Idea of Recursion

- A recursive solution has two parts:
    - **Base case** → stops the recursion
    - **Recursive case** → reduces the problem

PROF

Example:

```
int factorial(int n) {
    if (n == 0)
        return 1;          // base case
    return n * factorial(n - 1); // recursive case
}
```

Breaking Down Problems

- Recursion solves a problem by:
    1. Solving a smaller version of the same problem

2. Combining the result

Example idea:

```
factorial(5)
= 5 × factorial(4)
= 5 × 4 × factorial(3)
...
```

 Complex problem → sequence of simpler ones

Natural Problem Representation

- Some problems are **naturally recursive**

Examples:

- Factorial
- Fibonacci sequence
- Tree structures
- Nested data (folders, expressions)

 Recursion matches how we describe these problems

Cleaner and More Readable Code

Recursive version:

```
int sum(int n) {
    if (n == 0)
        return 0;
    return n + sum(n - 1);
}
```

Iterative version:

```
int sum(int n) {
    int s = 0;
    for (int i = 0; i <= n; i++)
        s += i;
    return s;
}
```

 Recursive code often mirrors the mathematical definition

Recursion and Data Structures

- Recursion is essential for:
    - Trees (binary trees, expression trees)
    - Graph traversal

Example idea:

```
process(node):
    process left child
    process right child
```

 Each part is handled the same way

Divide and Conquer

- Recursion is the basis of powerful algorithms:

Examples:

- Merge Sort
- Quick Sort

 Strategy:

1. Divide problem into parts
2. Solve each part recursively
3. Combine results

PROF

Backtracking and Exploration

- Recursion allows exploring multiple possibilities

Examples:

- Maze solving
- Sudoku
- Permutations

 Approach:

- Try a solution
- If it fails → go back (backtrack)
- Try another path

Recursion Uses the Stack

- Each recursive call creates a **stack frame**

```
f(3)
 → f(2)
   → f(1)
     → f(0)
```

- Then returns in reverse order

 Important for understanding memory usage

Advantages of Recursion

- Simplifies complex problems
- Matches mathematical definitions
- Natural for hierarchical structures
- Reduces code complexity (in some cases)

Limitations of Recursion

- Uses more memory (stack frames)
- Can be (slightly) slower than iteration
- Risk of **stack overflow**

 Must always include a correct base case

Recursion vs Iteration

PROF

Recursion	Iteration
Uses function calls	Uses loops
More expressive	Often more efficient
Uses stack memory	Uses less memory

When to Use Recursion

Use recursion when:

- Problem is naturally recursive
- Working with trees or graphs
- Using divide-and-conquer
- Code clarity is important

Summary

- Recursion = solving a problem using smaller instances of itself
- Requires:
    - Base case
    - Recursive case
- Powerful for:
    - Structured data
    - Complex problem solving

Heap Memory (Quick Introduction)

- Used for dynamic allocation

```
int *p = malloc(sizeof(int));
```

- Must be manually freed:

```
free(p);
```

Stack vs Heap

Feature	Stack	Heap
Allocation	Automatic	Manual
Speed	Fast	Slower
Lifetime	Function scope	Until freed
Size	Limited	Larger

PROF

Common Errors

- Using uninitialized variables
- Returning address of local variable
- Stack overflow (recursion)
- Memory leaks (heap)

Example error:

```c
int* func() {
    int x = 10;
    return &x; // WRONG
}
```

Summary

- Memory is divided into:
    - Global/static area
    - Stack
    - Heap
- Global/static → long lifetime
- Local → stack-based, temporary
- Stack manages function calls
- Recursion uses stack frames
- `extern` allows sharing global variables across files
- Heap allows dynamic allocation

Practice Exercises

1. Write a function using a static variable counter
2. Use `extern` to share a variable across two files
3. Trace stack execution of nested functions
4. Implement Fibonacci(n) recursively (careful! the trivial solution is inefficient)

PROF

Final Insight

 Understanding memory = understanding how your program really works
 Especially important for debugging, optimization, and advanced C programming
 Recursion turns **complex problems into simpler repeated steps**
 It is a key tool for thinking about algorithms and program structure

---